

Deep dive into Real-time Object Detection Methods

Project for CS-5814 Intro to Deep Learning

Sandeep Kollipara[†]

Department of Computer Science

Virginia Tech

Alexandria, VA, USA

sandeepk@vt.edu

ABSTRACT

Most security systems and sensor components require quick and standalone object detection methods that can warn the user in real-time directly or through downstream protocols. These real-time object detection methods generally distinguish from their general versions through low-latency responsiveness, lightweight build and relatively simple process. The focus of this project is to evaluate widely proclaimed deep learning models from R-CNN family and the YOLO series on images from dashcam footage of driving vehicles, BDD100K dataset.

The goal is to train 2 one-stage detectors (RetinaNet, YOLO-v3) and 1 two-stage detectors (Faster R-CNN FPN) with BDD100K dataset and evaluate bounding-box accuracy close to that of model benchmarks. This is an exercise in understanding neural network training regimens for deep learning models on real-world datasets. To this end, the PyTorch implementations for the 3 models are trained. An ablation study of hyperparameter tuning for YOLO-v3 is also conducted. The code for this project is available at: <https://github.com/sandeep-kollipara/ece5554-cs5814>

1 Introduction

Several applications of computer vision have been dominated by the sophisticated deep learning models and transformers that can ingest and generate multimodal data like audio, video, images, text, or their combinations. With the advent of attention and reasoning in LLMs, they have nearly unbounded scope in terms of ability in performing tasks that were traditionally handled by rule-based algorithms. However, the real-time applications have yet to be dethroned as the API call associated with the processing time of an LLM is high compared to the performance of the state-of-the-art real-time object detection techniques (at least for trained objects). Vision Language Models being generative models are more robust than the ‘simpler’ YOLO/R-CNN as they are flexible to detection of any object and can assist with visual question answering through their reasoning. However, their generalizability and complexity come with drawbacks for real-time application in the form of high computational and storage requirements. Enter the faster and relatively lightweight deep learning models like R-CNN and YOLO series which have withstood the brunt of AI advancements.

In this project, we focus mostly on a class of deep learning models called Convolutional Neural Networks (CNN) that work on special inputs like images and video and possess layers mimicking the filters/kernels used in image processing for feature extraction. The upside of using CNNs is the relatability and explainability of architecture in context of the problem which makes them an ideal starting point for students of deep learning.

2 (Proposed) Contributions

This project will explore the simpler deep learning models that made the breakthrough in the mid-2010s in real-time object detection. The models of interest are currently: 1) Faster R-CNN, 2) RetinaNet, 3) YOLO v3. These will be trained from scratch using the video datasets and bounding-box detection tested on the validation sets for accuracy metrics for the real world dataset: Berkeley Deep Drive BDD100K dataset of driving videos.

Two models from each family will be replicated manually for analyzing the intermediate outputs of layers within the deep neural network. These will be expanded into a separate section of results akin to ablation study but for the layers of the deep network. Any differences between the implementation and the manual build will be highlighted in the form of case study especially if related to optimization for real-time detection.

3 Related Work

Prior to the rise of deep learning models, there were traditional algorithms for real-time object detection which were widely used. One such method was the Viola-Jones detector (also known as Haar cascades method) introduced in 2001. It has been extensively used in cameras for face detection. Another is a feature descriptor from 2006 called Histogram of Oriented Gradients (HOG) detector which is flexible and similar to Scale-Invariant Feature Transform (SIFT) descriptors. In 2008, a more robust version of HOG detector by the name Deformable Part-based Model (DPM) detector was popular and used for detecting parts of objects. While the traditional methods are still used sparingly, they have been eclipsed by the revolution of the deep learning models. Unlike the traditional methods that use handcrafted features and rigid algorithms, the deep learning

architectures can retrieve representations from raw data directly (albeit at the cost of explainability).

The first successful deep learning model in real-time object detection was R-CNN in 2014 [7]. It led to subsequent development of Fast R-CNN [3] and Faster R-CNN [8]. All of them being two-stage detectors with the original utilizing CNN for feature extraction and SVM for classification. YOLO, a one-stage detector was introduced in 2015 by Joseph Radmon [5]. It utilizes grid prediction to detect objects in one-step and inspired other one-stage detectors like Single-shot Detector (SSD), RetinaNet and Fully Convolutional One-Stage object detector (FCOS). Until YOLO closed the gap in accuracy, the two-stage detectors significantly outmatched the previous one-stage detectors and continued still up until YOLO v3 and RetinaNet [6]. Since then, the YOLO series have retained the crown of state-of-the-art in the real-time object detection field with the latest v11 continuing to hold up well in industry and academic/research practices as well.

3.1 Motivation/Research Question

Real-time Computer Vision applications are on the lookout for more accurate, lightweight, faster and responsive deep neural network architectures whose problems will not be solved by simply increasing the parameters, training data or computational power like that of LLMs and VLMs. The better models are generated by improving the design using better understanding of the architecture and limited resources to the advantage. Also, most sensors and smart devices using real-time detection cannot rely on API calls over the network or internet to a remote server hosting the more complex deep learning models due to security concerns or latency issues. Understanding and improving these real-time models with the abovementioned constraints is a challenging task.

The objective for this project is to train deep neural networks using a real-world dataset for the both one-stage and two-stage object detectors. Can the models be trained to achieve an accuracy as close to that of benchmarks as possible?

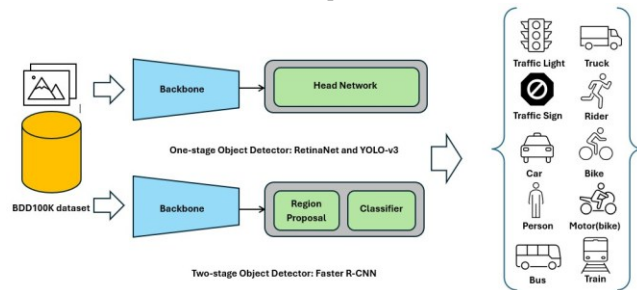


Figure 1: An illustration of the objective for the project: training one-stage and two-stage object detectors with BDD100K dataset of images with 10 distinct object classes

3.2 Problem Significance

Security and privacy are an evolving concern currently where access to unregulated technology is easy and surveillance

increasingly difficult. While installation of cameras is prevalent, their detection systems need to be updated periodically to keep up with the more innovative trespassers like drones. In these instances, it is necessary for detection systems to balance responsiveness and accuracy by negotiating tradeoff to perform optimally. Our literature survey has led us towards both traditional and machine learning algorithms with the latter further classified into (current state-of-the-art) one-stage and (older) two-stage object detection methods. For this project, we choose specifically from the machine learning category and apply it to our use cases of driving footage and drone detection. Our goal is to compare the different algorithms and weigh them against each other across factors, such as accuracy, responsiveness and resource consumption. The biggest challenge would be to root out noise and false positives such as insects, birds, airplanes, etc. flying at a recognizable distance to be mistaken for a drone while generalizing for different backgrounds.

4 Preliminaries

A basic question that arises from the problem is: why can't CNNs be directly used for object detection? CNNs were introduced in 1990s by LeCun et al and were replaced by SVMs shortly after until their re-emergence in 2012 at ImageNet Large Scale Visual Recognition Challenge (ILSVRC) with some addition to activation functions and dropout regularization. Soon after, the question asked was whether this could be applied for object detection as well? [7]

The answer is, that unlike image classification, object detection relies on the aspect of object localization (amongst other objects) within an image and training with smaller amount of annotated data [7].

5 Algorithm

To this end, the author of R-CNN had tested with sliding window approaches (like traditional algorithms) but failed. They instead adopt the 'recognition within region' paradigm where the method generates thousands of category-independent region proposals for a single image input and extracts a feature vector for each region. This feature vector is then classified at region-level using a category-specific linear SVM. Since their method combines CNN with regional proposals, it is dubbed R-CNN for Regional-CNN. To rectify the errors from localization despite the above methodology, the authors introduced the bounding box regression method on top to create R-CNN BB that reduces mislocalizations.

SPPNet [4] was introduced shortly afterwards to address the fixed scale problem of R-CNNs. This was addressed using a pooling strategy by the name 'Spatial Pyramid Pooling' where the network structure can generate a fixed length feature vector regardless of the input size. This allowed the model to acquaint with images of any scale. They also speed up R-CNN by sharing computation.

Fast R-CNN [3] tackles broader drawbacks such as: slower training, high training memory plus time and slow detection. This is in tandem with two of R-CNN's shortcomings regarding object localization: multiple candidate objects across regions rough localization. So, the author trained a VGG16 (very deep network) and utilized SPPNet for computation sharing for quicker training and faster processing of features. However, it has its share of drawbacks as well: multi-stage pipeline for training and unable to finetune before SPPNet.

Faster-CNN [8] combats the test-time bottleneck of its predecessor, namely the region proposal method which are implemented in CPU for research unlike GPU for region-based calculations. This is addressed with an algorithmic change by introduction of Regional Proposal Networks (RPN) which share convolutional layers with the object detection networks.

You Only Look Once (YOLO) [5] is a different approach from the rest where the object detection is framed as a regression problem. The predictions are for spatially separated bounding boxes as well as class probabilities in one step (unified detection). At the onset, YOLO is extremely fast and predicts less false positives but does make mistakes in localizations. Unlike (traditional) DPM detector which uses a sliding window approach and R-CNN using regional proposals to generate bounding box (followed by classifier), YOLO simply predicts both simultaneously using a single network. It is fast and does not require sliding window approach as it contextualizes all parts of the image (reducing background errors). However, it lags in accuracy compared to the state-of-the-art at that time.

YOLO v2 dubbed YOLO9000 [2] (for detecting over 9000 classes) is better than Faster R-CNN, ResNet and SDD. It improves upon v1's shortfalls of low recall and localization errors by simplifying itself further instead of scaling up (like most deep networks). The improvements were made due to: 1) Batch Normalization: leads to convergence while eliminating other forms of regularization (even remove 'dropout' without overfitting), 2) High Resolution Classifier: unlike previous iterations, this model is trained on high resolution while learning object detection. In addition, several other improvements were made like: 3) Convolution with Anchor Boxes, 4) Dimension Clusters, 5) Direct Location Prediction and 6) Multi-scale Training. While the above refinements increased the mAP slightly, the resource consumption fell by 33%.

YOLO v3 was the last version to be worked on directly by the original author Joseph Redmon. All the versions following this were mostly engineering updates and not published as research output. YOLO v3 combines FPN (Feature Pyramidal Networks) to enable prediction at multiple scales: Multi-scale training. It introduces residual architecture of DarkNet53 compatible with GPU hardware resulting in faster inference times. Label assignment was also revised for anchor boxes. Spatial Pyramid Pooling (SPP) assigned multiple maximum pooling layers with a stride of 1 across local to global kernel sizes allowing each grid to obtain multiple resolution features from local to global. This has

proven to greatly improve accuracy[6]. Also, YOLO v3 happens to be the most popular YOLO version.

RetinaNet (alongside YOLO v3) was the first single-stage object detectors to exceed the accuracy of two-stage Faster R-CNN and have since led the road for one-stage detectors to be state-of-the-art. RetinaNet achieves this through the introduction of focal loss function to address extreme class imbalance in dense prediction scenarios.

5.1 Approach

Planning: Instead of the latest versions, the chosen implementations were contemporary to when the one-stage and two-stage detectors were closer in accuracy. To that end, Faster R-CNN FPN (Feature Pyramidal Networks) was the chosen two-stage detector from Detectron 2's implementation. RetinaNet and YOLO-v3 were chosen as the one-stage detectors but across Detectron 2 and a custom PyTorch implementation respectively. We first tested using the test datasets they were benchmarked with to gather an understanding of the configurations and importantly, the format of annotations.

Data Engineering: The dataset came with annotations for not just bounding box detection but also semantic segmentation, keypoint detection and panoptic segmentation. These were not in the requisite formats for either Detectron 2 or custom PyTorch implementations and had to be processed into the standard COCO 2017 format for Detectron 2 and a custom format for the other implementation. The Python code developed for this conversion can be found in the repo.

6 Data Description

The real-world dataset called Berkeley Deep Dive, shortened as BDD100K contains 100,000 images of driving video taken across 4 cities in USA. Originally a video dataset containing 40 second clips of 100,000 different instances of which the 10th second is retrieved as an image and annotated for 10 classes of objects for various tasks mentioned before. Out of 100,000 images, 70,000 in train set and 10,000 in validation set are annotated with the remaining 20,000 as unlabeled test set.



Figure 2: A histogram depicting the frequency distribution of various object classes in BDD100K dataset. The y-axis being exponential in scale highlights the class imbalance in the data.

Data augmentation is conducted by default in YOLO-v3 and Detectron 2 and does not need to be conducted manually.

6.1 Data Engineering

This section is dedicated for the data engineering involved in preparing the BDD100K dataset annotations for both the implementations: Detectron 2 and Custom PyTorch (YOLO-v3).

The Detectron 2 implementation from Facebook Research (now Meta) incorporates several visual recognition techniques ranging from bounding box detection, semantic segmentation, keypoint detection and panoptic segmentation. It uses COCO 2017 for benchmarking its models. COCO 2017 is a large-scale object detection, segmentation and captioning dataset containing 330,000 images (over 200,000 labelled) with 80 object categories (classes). Its labels for the images are annotated in a standardized JSON notation that is consistent across all of Detectron 2's models. However, the labels for our test dataset of BDD100K has its own JSON notation that is incompatible with that of Detectron 2's COCO 2017 format.

For the Custom PyTorch implementation of YOLO-v3, the situation is similar except it uses COCO 2014 instead. Also, there are no JSON annotations for input but instead a directory of text files reflecting that of the image files, containing the tab-delimited tabular data for the image-level bounding box annotations.

Since both implementations require conversion of BDD100K dataset's annotations to be usable as input, the BDD100K's JSON was parsed to extract all levels of data using Python's Pandas and JSON packages. They were then filtered for only the bounding box coordinates while ignoring the remaining labels from instance/panoptic segmentation and keypoint detection. This was conducted for both train and validation set annotations.

The bounding box inputs were the diagonal vertex coordinates in the case of BDD100K. Both the implementations required different transformation of coordinates to be viable for use. The Detectron 2's COCO 2017 format required the closest and farthest diagonal coordinates plus the width and height. The Custom PyTorch implementation required the box-center coordinates plus the width and height instead. In addition, they also needed to be minmax-normalized between 0 and 1.

Both implementations also required the labels to be indexed and id-ed accordingly within the annotations. These were thoroughly QA-tested manually after being generated by the Python scripts. This section took the most time of the project due to the size of the datasets involved, the time taken to study the implementations and analyze the standard annotation formats as well as the test dataset's annotations.

7 Experiments

The three selected methods: 1) Faster R-CNN, 2) RetinaNet, 3) YOLO v3, will be trained on the BDD100K dataset using near identical training protocol. All 3 will be tested against the benchmark metrics of accuracy (Average Precision). The hyperparameter tuning required for reaching the best results will be discussed in an ablation study.

Baselines: Originally, the Histogram of Oriented Gradients (HOG) was considered as a baseline being a traditional detection method. The open-source implementations required extensive effort for supporting multi-class detection and the alternatives required MATLAB. Instead, it was decided to test on two instances of each of the one-stage and two-stage object detectors that serve as comparison for one another. For one-stage detectors: RetinaNet and YOLO-v3. For two-stage detectors, Faster R-CNN with ResNet50-FPN backbone. We expect the newer YOLO-v3 to perform better than RetinaNet and both should ideally surpass Faster R-CNN. For consistency across the board, we train all models on backbones with approximately 50 hidden layers.

System: Following are the systems used for training the models:

- **Detectron 2:** RetinaNet and Faster R-CNN on Google Colab with T4 (15GB vRAM) and A100 (40GB vRAM) GPUs
- **Custom PyTorch:** YOLO-v3 on AMD PC with Nvidia 5070Ti (12GB vRAM)

Hyperparameters: Following regimen of training was followed for each implementation:

- **Detectron 2:** The following were maintained consistent across the Faster R-CNN and RetinaNet models
 - Input size: 800-1333 x 800-1333 (adjusts to input image)
 - Batch size: 8
 - Learning rate: 0.0001 (initial) >> reduce to 30% after every cycle of 3000, 5000, 7000 and 9000 iterations
 - CLS Threshold (Confidence): 0.7 (default)
 - IoU Threshold (Area of overlap): 0.5 (0.75 also calculated)
 - Iterations: 38,000 iterations in total
- **Custom PyTorch:** The following was maintained for YOLO-v3:
 - Input size: 416 x 416 (standard) >> 640 x 640 (custom)
 - Batch size: 8 (standard) >> 6 (custom, GPU vRAM constraint)
 - Learning rate: 0.0001 (initial) >> reduce to 30% after every cycle of 5, 7, 9, 11 and 13 epochs
 - CLS Threshold (Confidence): 0.001 (default)
 - IoU Threshold (Area of overlap): 0.5 (standard)
 - NMS (Non Maximum Suppression) Threshold: 0.4 (standard)
 - Epochs: 60 epochs of the dataset
- **Learning Rate Schedule:** It has been observed that increasing learning rate after successive decrements can help escape local minima and achieve higher accuracies. This was tested with a higher LR of 0.003 after 3 cycles of decrements and it resulted in a higher mAP for RetinaNet and Faster R-CNN with the former benefitting much more.

Benchmarks: Tabulated below are the benchmarks used:

Table 1. Benchmark of the models used in the project.

COCO Version	Model	Backbone	mAP %
2017	Faster R-CNN	ResNet50-FPN	40.2
2017	RetinaNet	ResNet50	38.7
2014	YOLO-v3 paper (416)	DarkNet53	55.3
2014	YOLO-v3 impl. (416)	DarkNet53	55.5
2014	YOLO-v3 paper (608)	DarkNet53	57.9
2014	YOLO-v3 impl. (608)	DarkNet53	57.3

Evaluation Metrics: The accuracy for object detection is described by Average Precision (AP) metric which is calculated for each class using the classification metric of Precision. The mean Average Precision (mAP) is calculated as the mean of AP across all the classes. This metric is reliant on the positive predictive power of the model (false positives) and not on the ground truth.

7.1 Results

Following are the challenges that affected the outcome or forced us to explore alternative hyperparameters:

Table 2. The Mean Average Precision metrics across the 3 models for the BDD100K dataset.

Class	Faster R-CNN (R50-FPN)	RetinaNet (R50)	YOLO-v3 (DarkNet53)
Traffic Light	13.970	15.751	54.199
Traffic Sign	20.528	25.854	51.059
Car	37.353	42.136	69.672
Person	23.963	28.689	48.266
Bus	21.370	35.860	50.894
Truck	20.157	35.024	51.405
Rider	3.949	11.328	34.922
Bike	12.800	16.748	30.852
Motor	3.823	14.004	24.302
Train	0.000	0.000	0.000
mAP %	27.435	41.648	41.577

Inference: Both one-stage detectors have reached a respectable mAP level of 40% which is just 10-15% shy of the benchmark. The same could not be said for the two-stage detector which is quite low in mAP in comparison to the one-stage counterparts. Also, we see that there are excess predictions for RetinaNet and Faster R-CNN compared to YOLO-v3 suggesting that they may need to be filtered more strictly. Overall, the numbers are close to expectations for the objective at hand.

7.2 Ablation Study

We compare the impact of 2 different resolutions of input sizes for YOLO-v3 which made a huge difference in the final accuracy after successive training cycles. For the standard input size of (416 x 416), we only reached 11% mAP after 2 days of training. However, the accuracy was able to jump to 40%+ upon increasing the input size to (640 x 640) which allowed more of the smaller bounding boxes to be detected. Below are collection of plots tracing the failed (11% mAP) and successful runs (41% mAP):

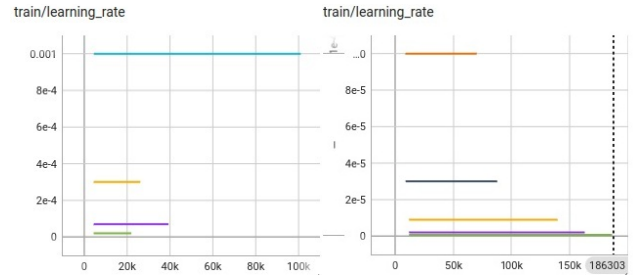


Figure 3(a): Learning rate schedules for the failed run (left) and successful run (right)

We follow a learning rate regiment that starts with a high initial rate of 0.001 and train it for 5 epochs until the loss rate stabilizes. We then iteratively reduce it by ~30% and repeat the process with increasing epoch counts of 7, 9, 11 and 13. This schedule basically helps in the convergence at a local minima with a confirmed increase in training accuracy. While this was true for the successful run, the learning rate was tested for fewer epochs in successive iterations for the failed run as there was considerable doubt if the learning rate was set correctly at that point in time.

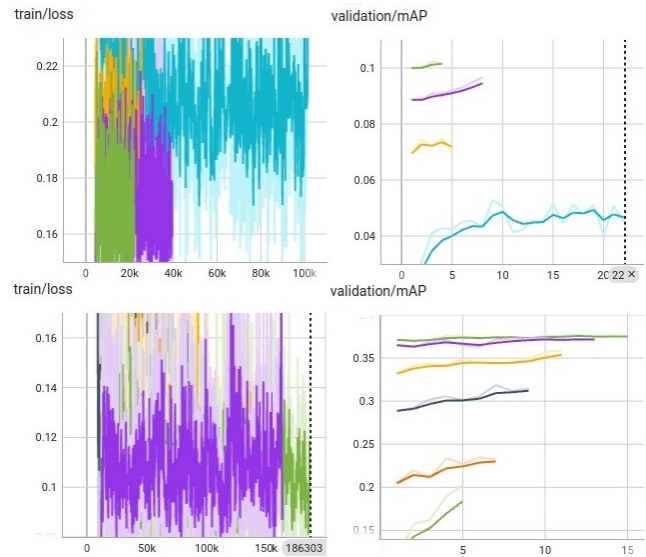


Figure 3(b): Training Loss and Validation Accuracy (mAP) for the failed run (top) and successful run (bottom)

However, training accuracy alone is not a good indicator of model performance, and it should be accompanied by a corresponding increase in validation accuracy too. This was achieved by the hyperparameter optimization via input size resolution for YOLO-v3 as demonstrated by these plots.

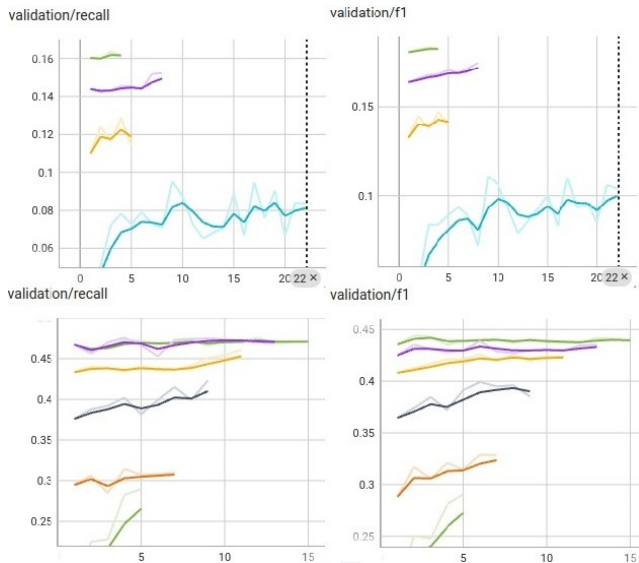


Figure 3(c): Training Loss and Validation Accuracy (mAP) for the failed run (top) and successful run (bottom)

The plots depict a sustained increase in performance metrics and decrease in training loss that diminishes with time for a set learning rate. This effect is seen to be less prominent for the next iteration from the last. The mAP metric is also accompanied by the corresponding Precision and Recall metrics following a similar trend.

This marked difference in performance across input sizes confirms that the labels for the BDD100K are smaller than the optimal detection size for YOLO-v3. Increasing the resolution is accompanied by an increase in overall size of the neural net from the start to finish. This necessitates an increase in GPU's storage (vRAM) to accommodate the higher resolution images. However, most devices have limited storage especially PCs which in turn require us to reduce the batch size. This leads to higher training times and slower learning which can be quite costly while training datasets of this scale.

We also considered jolting the learning rate to 0.003 (higher than normal) to escape the local minima (if the model is stuck). This backfired for YOLO-v3 as the mAP reduced to below 20% after training for a few epochs. This however was very beneficial in the case of Detectron 2 where the mAP increased from higher 20s to 40%+. While this can be studied separately as a different ablation study, the results were purely supplementary. This has been discussed again under the 'Challenges Faced' section with additional information relevant to the implementations.

8 Discussion

The project was originally ambitious in its objectives regarding architecting a model from scratch and testing on two datasets. At that time, the training times for models were not thoroughly researched. It was recommended that around 300 epochs of training be conducted on YOLO-v3 for the best performance and a time period of 5 days using home computer systems was suggested by the developers and netizen users. This was severely underestimated during the project proposal as our final model only underwent 60 epochs of training.

Some future work can include ways to mitigate class imbalance using weighted loss functions and test more hyperparameters and backbones for the models.

It was also originally proposed that traditional methods like Histogram of Oriented Gradients (HOG) or Deformable Parts Models (DPM) be tested but was dropped due to workload constraints.

8.1 Challenges faced

Following are the challenges that affected the outcome or forced us to explore alternative hyperparameters:

1. **Small bounding boxes:** Many of the annotations in the BDD100K dataset contain significantly smaller bounding box instances with respect to its resolution of 1280 x 720
 - The consecutive convolution and pooling layers reduce the image information so much so that some labels might not have minimum detection supported by the models
 - YOLO-v3 can detect as low as 16 x 16 pixels in an input image (416 x 416) but even this may be insufficient when resizing 720p images from BDD100K dataset
2. **Imbalanced dataset:** Being a real-world dataset, the learning of the neural network as affected by the frequency of the objects by class
 - Class 'Train' is not detected at all due to its low frequency
 - Research to tackle imbalanced classes within images for object detection is still ongoing
3. **Curse of Local Minima:** A general optimization problem in neural networks where convergence occurs at a suboptimal state and lowering the learning rate does not improve the validation accuracy
 - This is navigated by managing learning rate schedules
 - Decrease from initial rate until convergence, then reset to a higher learning rate and repeat
4. **Custom Evaluator Function:** Detectron 2 has issues with excess prediction being generated for RetinaNet
 - This was bypassed through an additional filter added to the code referred in an issue from the GitHub page

- However, the predictions seem to have no Class threshold (CLS threshold) applied and predictions with low confidence were also evaluated

5. Long training times:

- On a NVidia GPU for a PC, it took 4 days to train YOLO-v3 for the successful run
- On a T4 GPU considered standard on Google Colab, it took 2 days of training to reach the accuracy mentioned in results
- At the end, Colab's A100 GPU was used which was 15x faster than T4 to get the last set of results in few hours.

9 Conclusion

The report has tested the real-world driving video dataset against 3 models of deep real-time object detection: Faster R-CNN, YOLO-v3 and RetinaNet. We tested different resolutions for YOLO-v3 and achieved good results. In the other cases, we escaped local minima by managing learning rates manually. This worked really well for RetinaNet's outcome. At the end, our results were consistent with the historical context while also being close to benchmark for 2 out of 3 cases.

ACKNOWLEDGMENTS

This is a cross-course project satisfying the project components across both 'CS 5814 - Intro to Deep Learning' and 'ECE 5554 - Computer Vision' offered in Fall 2025. The scope of the project's goals and results was identical for both the courses although the deliverables were different. While the ECE 5554 necessitated formation of teams, the contribution of the other members was limited. A website for this report's counterpart for ECE 5554 can be found at <https://sites.google.com/view/vt-ece-5554-fall-2025-group-22/>.

REFERENCES

1. Dollár, T.-Y. L. (2018). Focal Loss for Dense Object Detection. *arXiv*, 1708.02002.
2. Farhadi, J. R. (2016). YOLO9000: Better, Faster, Stronger. *arXiv*, 1612.08242.
3. Girshick, R. (2015). Fast R-CNN. *arXiv*, 1504.08083.
4. He, K. a. (2014). Spatial Pyramid Pooling in Deep Convolutional Networks for Visual Recognition. *Computer Vision – ECCV 2014 Springer International Publishing*, 1611-1624.
5. Joseph Redmon, S. D. (2016). You Only Look Once: Unified, Real-Time Object Detection. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 779-788.
6. Liao, C.-Y. W.-Y. (2024). YOLOv1 to YOLOv10: The fastest and most accurate real-time object detection systems. *arXiv*, 2408.09332.
7. Malik, R. G. (2014). Rich feature hierarchies for accurate object detection and semantic segmentation. *arXiv*, 1311.2524.
8. Sun, S. R. (2016). Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. *arXiv*, 1506.01497.